MongoMeals

Module-Wise FSD Project Explanation

Simple, easy and faculty-ready viva guide

Project in one line

MongoMeals is a MERN-stack restaurant management application with Customer and Admin roles. React creates the interface, Node and Express handle APIs and logic, MongoDB stores data, and Mongoose connects the backend with MongoDB.

Best presentation method

Explain one module at a time in this order: What the page does -> which code runs -> what logic is applied -> what data is stored in MongoDB. Do not explain all files together.

1. Project Introduction

Ready-made introduction

“My project name is MongoMeals. It is a full-stack restaurant management system developed using the MERN stack. It has two roles: Customer and Admin. Customers can view food, place orders, reserve tables, request events, submit reviews and use rewards. The Admin can manage menu items, orders, reservations, reviews, events, users and rewards.”

Technology stack

- React: creates the frontend pages and components.
- Node.js: runs JavaScript on the backend.
- Express.js: creates routes and APIs.
- MongoDB: stores permanent project data.
- Mongoose: connects Node.js to MongoDB and defines data models.
- JWT: identifies a logged-in user.
- bcrypt: converts passwords into secure hashes.
- Multer: uploads menu and reward images.
- Nodemailer: sends emails.
- Fetch API: sends requests from React to the backend.

Common flow used in every module

User clicks or submits something
↓
React sends an HTTP request
↓
Express route receives it
↓
Middleware checks login/role
↓
Controller performs the logic
↓
Mongoose reads or writes MongoDB
↓
Backend returns JSON
↓
React updates the page

2. Home Module

What to say to faculty

“The Home page introduces the restaurant and gives navigation to Menu, Reservation, Rewards, Events, FAQ and Sign-in. It mainly displays information and redirects the customer to other modules.”

Main code

Frontend: src/pages/Home.jsx, src/components/Navbar.jsx, src/App.jsx

Backend: Mainly React Router; no special data controller is required for normal static sections.

MongoDB: Usually no separate collection for static Home content.

Working flow

Customer opens Home page
↓
React renders the Home component
↓
Customer clicks a navigation button
↓
React Router opens the selected page

Core logic

- React Router changes pages without reloading the entire website.
- The Home page is mainly a presentation and navigation module.

Faculty may ask

Question: “Does Home store data?” Answer: “Normally no. It mainly displays static information and links to other modules.”

3. Menu and Cart Module

What to say to faculty

“The Menu module reads food items from MongoDB and displays each item as a card. A customer can add food to the cart, change quantity and place an order. The Admin can add, edit or delete menu cards.”

Main code

Frontend: src/pages/MenuPage.jsx, src/context/CartContext.jsx, Cart components

Backend: server/routes/menuRoutes.js, server/controllers/menuController.js, server/models/MenuItem.js

MongoDB: menuitems collection; orders collection after checkout

Working flow

Menu page opens
↓
GET /api/menu is called
↓
Controller reads menu items
↓
MongoDB returns an array
↓
React uses map()
↓
One card is created for each food item

Core logic

- map() repeats the same card design for every menu item returned from MongoDB.
- If the same food is added again, quantity increases instead of creating a duplicate cart row.
- Subtotal = price x quantity. GST and service fee are then added.
- The cart is managed with React Context so it can be used on different pages.

Faculty may ask

Question: “How does a new card appear?” Answer: “The Admin saves a new MenuItem document. When the customer page calls GET /api/menu again, React maps that new document into a new card.”

4. Table Reservation Module

What to say to faculty

“The Reservation module lets a logged-in customer select a date, time, guest count and table. The backend prevents past bookings and duplicate table bookings.”

Main code

Frontend: src/pages/Reservation.jsx

Backend: server/routes/reservationRoutes.js, server/controllers/reservationController.js, server/models/Reservation.js

MongoDB: reservations collection

Working flow

Customer selects date and time

↓

Frontend asks for occupied tables

↓

Backend checks reservations

↓

Occupied table numbers are returned

↓

Customer selects an available table

↓

POST /api/reservations is sent

↓

Backend validates and saves the booking

Core logic

- Past dates and past times are rejected.
- Restaurant working hours are checked.
- Duplicate check uses the same date, same time and same table.
- Cancelled reservations do not block a table.

Faculty may ask

Question: “Can two users book Table 4?” Answer: “Yes, but not for the same date and time. The backend checks MongoDB before saving.”

5. Rewards Module

What to say to faculty

“The Rewards module increases customer engagement. Customers earn points from registration, reservations, approved reviews and orders. They can redeem points for active rewards.”

Main code

Frontend: src/pages/PrestigeRewards.jsx and reward-related components

Backend: server/routes/rewardRoutes.js, server/controllers/rewardController.js, User, Reward and RewardHistory models

MongoDB: users, rewards and rewardhistories collections

Working flow

Customer performs an eligible action

↓

Backend calculates points

↓

User balance is updated

↓

RewardHistory record is created

↓

Customer sees the new balance

Core logic

- Registration gives 50 points.
- The first reservation can give a first-booking bonus plus normal reservation points.
- An approved review gives points only once.
- Before redeeming, the backend checks whether the user has enough points.
- After redemption, points are deducted and history is stored.

Faculty may ask

Question: “Why use RewardHistory?” Answer: “The User document stores the current balance, while RewardHistory stores when and why points were earned or spent.”

6. Events Module

What to say to faculty

“The Events module allows customers to request birthdays, anniversaries, corporate functions or private events. The request is saved as Pending, and the Admin later contacts and confirms it.”

Main code

Frontend: src/pages/Events.jsx

Backend: server/routes/eventRoutes.js, server/controllers/eventController.js, server/models/EventRequest.js

MongoDB: eventrequests collection

Working flow

Customer fills event form

↓

POST event request is sent

↓

Backend validates the fields

↓

MongoDB stores the request as Pending

↓

Admin views the request

↓

Admin changes its status

Core logic

- Common statuses are Pending, Contacted, Confirmed and Cancelled.
- The event form stores customer, date, guest count, event type and requirements.

Faculty may ask

Question: “Is an event automatically confirmed?” Answer: “No. It is first saved as Pending. The Admin reviews and updates the status.”

7. FAQ Module

What to say to faculty

“The FAQ module provides quick answers to common questions about booking, orders, rewards and services. When the user clicks a question, React opens or closes its answer.”

Main code

Frontend: src/pages/FAQPage.jsx

Backend: Normally no backend route is needed if FAQs are static.

MongoDB: No collection is required for static FAQ data.

Working flow

Customer clicks a question

↓

React changes the selected/open state

↓

The answer becomes visible

Core logic

- This is mainly frontend state logic.
- If FAQs are hardcoded, MongoDB is not required.

Faculty may ask

Question: “Why is MongoDB not used here?” Answer: “Because the questions and answers are directly written in the frontend code.”

8. Sign-up, Sign-in and Logout

What to say to faculty

“Authentication allows users and Admins to securely access protected functionality. Passwords are protected with bcrypt, and login identity is maintained using a JWT token.”

Main code

Frontend: src/pages/AuthPage.jsx, src/context/AuthContext.jsx, src/services/api.js

Backend: server/routes/authRoutes.js, server/controllers/authController.js, server/models/User.js, auth middleware

MongoDB: users collection

Working flow

User submits email and password

↓

Backend finds the email

↓

bcrypt checks the password

↓

JWT token is created

↓

Token and user data return to React

↓

Token is stored in localStorage

↓

Protected requests send the token

Core logic

- Registration first checks whether the email already exists.
- The original password is not stored; bcrypt stores a hash.
- The same login form can be used for both roles.
- The role field decides whether the account is a normal user or Admin.
- Logout removes the token from localStorage.

Faculty may ask

Important: A customer does not become an Admin after login. Only an account whose role is “admin” can access the Admin dashboard.

9. Customer Dashboard

What to say to faculty

“After a customer signs in, the dashboard shows only that customer’s profile, orders, reservations and reward history.”

Main code

Frontend: src/pages/Dashboard.jsx

Backend: user, order and reservation routes/controllers

MongoDB: users, orders, reservations and rewardhistories

Working flow

Dashboard sends protected API requests

↓

JWT middleware identifies the customer

↓

Backend searches using that user ID

↓

Only matching records are returned

↓

React displays personal history

Core logic

- The user ID comes from the verified JWT.
- Orders and reservations are filtered by the logged-in user ID.
- One customer cannot normally see another customer's records.

Faculty may ask

Question: "How does the backend know which customer?" Answer: "The JWT contains the user ID. After verification, the middleware sets req.user."

10. Admin Dashboard

What to say to faculty

"The Admin dashboard is the management area. It allows an Admin to manage users, menu cards, orders, reservations, reviews, rewards and event requests."

Main code

Frontend: src/pages/AdminPage.jsx, AdminRoute component

Backend: server/routes/adminRoutes.js, server/controllers/adminController.js, auth/admin middleware

MongoDB: Uses all major project collections

Working flow

Admin logs in

↓

JWT identifies the account

↓

Admin middleware checks role

↓

Admin APIs are allowed

↓

Dashboard loads all management data

Core logic

- Frontend AdminRoute hides the page from normal users.
- Backend admin middleware gives the real security.

- Admin can create, read, update and delete appropriate data.
- Passwords should be excluded when Admin user lists are returned.

Faculty may ask

Question: “Why check Admin in both frontend and backend?” Answer: “Frontend improves navigation, but backend security stops a normal user from directly calling an Admin API.”

11. Admin Management - Quick Explanation

Admin section	What Admin does	Main data
Menu	Add, edit, delete food cards and upload images.	MenuItem model / menuitems
Orders	View customer orders and their statuses.	Order model / orders
Reservations	View customer date, time, table and guest count.	Reservation model / reservations
Reviews	Approve or reject reviews; approved review can give points once.	Review model / reviews
Rewards	Add, edit, delete, activate or deactivate rewards.	Reward model / rewards
Events	View requests and change Pending to Contacted or Confirmed.	EventRequest / eventrequests
Users	View customer details without sending password hashes.	User model / users

12. MongoDB Compass and Data Storage

Faculty-ready explanation

“MongoDB is the actual database, while MongoDB Compass is a graphical tool used to view and manage that database. Mongoose connects the Node.js backend with MongoDB.”

Connection example: mongodb://127.0.0.1:27017/mongo-meals

- 127.0.0.1 means my own computer.
- 27017 is MongoDB’s default port.
- mongo-meals is the database name.
- process.env.MONGO_URI reads the connection URL from the .env file.
- If an environment variable is not available, the project may use a local fallback URL.

Controller calls a Mongoose model

↓

Mongoose sends a query to MongoDB

↓

MongoDB creates, reads, updates or deletes a document

↓

Result returns to controller

↓

Controller sends JSON to React

13. Small Viva Questions - API, Token and Other Terms

Term	Small and easy meaning
API	API means Application Programming Interface. In this project, it is the communication bridge between React and the Node/Express backend.
Endpoint	An endpoint is a specific API address, such as /api/menu or /api/auth/login.
Route	A route connects an HTTP method and URL to a controller function.
Controller	A controller contains the main business logic, such as login checking, reservation validation or reward calculation.
Model	A model defines what fields a MongoDB document can store.
Mongoose	Mongoose is the library that connects Node.js with MongoDB and provides models and database queries.
Token	A token is a digital proof that the user has already logged in.
JWT	JWT means JSON Web Token. It carries user identity information and is signed so the backend can verify it.
JWT three parts	Header, Payload and Signature. Header tells the token type, Payload contains data such as user ID, and Signature proves the token was not changed.
Bearer token	Bearer means the person carrying a valid token is allowed to access the protected API. It is sent as Authorization: Bearer TOKEN.
protect middleware	It reads and verifies the JWT, finds the user, and places that user in req.user.
Admin middleware	It checks whether req.user.role is admin before allowing Admin functionality.
bcrypt	bcrypt converts a password into a one-way secure hash. The real password is not directly stored.
localStorage	localStorage is browser storage. This project stores the JWT there using a key such as mongomeals-token.
Cookie	A cookie is a small value stored by the browser and automatically sent with matching requests.
Session	A session usually stores login data on the server, while the browser keeps only a session ID cookie.
Does this project use sessions?	No. The current project uses JWT plus localStorage, not traditional express-session authentication.
30-day token expiry	After 30 days the JWT becomes invalid and the user must log in again. The account, rewards and MongoDB data are not deleted.

Fetch API	fetch() sends HTTP requests from the React frontend to the backend.
Axios	Axios is another library for sending API requests. This project uses fetch(), not Axios.
async	async means the function performs asynchronous work and can use await.
await	await pauses that function until a Promise finishes, such as waiting for an API response.
JSON	JSON is a simple text format used to exchange data between frontend and backend.
FormData	FormData is used when sending files, such as a menu image, along with text fields.
Content-Type: application/json	It tells the backend that the request body contains JSON data.
Why not set JSON for FormData?	The browser must automatically create the multipart boundary for file uploads, so the code does not manually set application/json.
response.ok	It is true when the HTTP response status indicates success.
204 No Content	The request succeeded, but the server returned no response body. Therefore response.json() should not be called.
GET	Used to read data.
POST	Used to create new data.
PUT	Used to update existing data.
DELETE	Used to delete data.
200	Request completed successfully.
201	New data was created successfully.
204	Success with no returned content.
400	Invalid input or bad request.
401	User is not authenticated or token is invalid.
403	User is logged in but does not have permission.
404	Requested data was not found.
500	Backend server error.
process.env.MONGO_URI	It reads the MongoDB connection address from the environment file instead of hardcoding it in the source code.
.env file	It stores configuration and sensitive values such as database URLs, JWT secrets, email credentials and ports.
CORS	CORS allows the React frontend and Node backend to communicate when they run on different ports or origins.

express.json()	It converts incoming JSON request data into req.body.
Multer	Multer accepts uploaded files and stores them in the uploads folder.
Nodemailer	Nodemailer sends emails from the Node.js backend.
React Context	Context stores shared data such as login state or cart items so multiple components can use it.
React Router	React Router changes pages in the single-page application without reloading the complete website.
CRUD	Create, Read, Update and Delete - the four common database operations.
map()	map() repeats a UI design for every item in an array, such as creating one card per menu item.
filter()	filter() creates a new array containing only matching items, such as removing one cart item.
find()	find() returns the first array item that matches a condition.
populate()	populate() replaces a stored related ID with selected data from the related MongoDB document, such as user name and email.
_id	MongoDB automatically creates a unique _id for every document.
$ne	$ne means not equal. It can be used to ignore Cancelled reservations.
Date.now()	It returns the current timestamp and may be used as a simple unique numeric value.

14. Your apiFetch Code - Easy Meaning

Purpose

apiFetch is one common function used by the frontend to call different backend APIs. It adds headers, sends the request, handles errors and converts successful JSON responses.

```
if (!(options.body instanceof FormData)) headers['Content-Type'] = 'application/json';
```

- If the body is normal data, tell the backend it is JSON.
- If the body is FormData, do not manually set JSON because it may contain an image file.

```
const response = await fetch(url, { ...options, headers });
```

- Send the request and wait for the backend response.

```
if (!response.ok) { ... throw new Error(...) }
```

- If the backend returns an error status, read the message and throw an error.

```
if (response.status !== 204) return response.json();
```

- If the response is not 204, convert the returned JSON into JavaScript data.
- If it is 204, return nothing because “204 No Content” has no response body.

15. Two-Minute Final Presentation Script

Speak this at the end

“My project is MongoMeals, a MERN-stack restaurant management system with Customer and Admin roles. The customer can browse menu cards, manage a cart, place an order, reserve a table, request an event, submit a review and use reward points. The Admin can manage food cards, users, orders, reservations, reviews, rewards and event requests.

When a user performs an action, React sends an API request using fetch(). Express routes receive the request, middleware checks the JWT and role, and the controller performs the main logic. Mongoose then reads or writes data in MongoDB, and the backend returns JSON to React.

Authentication uses bcrypt and JWT. bcrypt protects passwords, and JWT identifies the logged-in user. The token is stored in localStorage and expires after 30 days; only the login expires, not the account or rewards. MongoDB Compass is used to view collections such as users, menuitems, orders, reservations, reviews, rewards and eventrequests.”

16. Best Viva Rule

Remember this pattern

For every module say only: 1) what the customer/Admin can do, 2) which React page sends the request, 3) which route/controller performs the logic, and 4) which MongoDB collection stores the result.